

Building CineIQ: How I Built a Movie Recommender You Can Actually Trust

Ayush Gahlot

Indian Institute of Technology, Guwahati

May 2026

Abstract

As streaming platforms have grown, their movie suggestions have become confusing. We often get stuck in loops, seeing movies the platform wants to push rather than what we actually want to watch. CineIQ is a project born out of frustration with these confusing systems. This report details my journey of building a clear and honest movie recommendation engine from scratch. It covers the struggles of working with limited computer memory, mixing different ways to find good movies, and using a custom AI to understand real audience feelings. The final system does not just guess what you want to watch; it tells you exactly *why* it picked that movie.

1 Introduction: The Problem with Hidden Algorithms

Ever wondered why a streaming service recommends a terrible movie just because it has one actor from your favorite film? Modern recommendation engines are great at keeping you clicking, but they are terrible at explaining their choices. When we stop trusting the suggestions, we stop enjoying the app.

The goal with CineIQ was simple but hard to build: create an engine that finds movies you will actually enjoy, and be 100 percent honest about how it found them. To do this, I needed to go beyond basic matching. I needed a system that understood your watching habits, the details of the movies, and most importantly, how real people felt about them.

This report outlines how I built CineIQ from start to finish. It details the problems I faced, like handling a dataset of 25 million ratings without crashing my computer, and the choices I made to solve them.

2 Phase 1: Managing Huge Amounts of Data

Every good recommendation engine starts with data. For CineIQ, I chose a massive dataset of 25 million user ratings and combined it with a detailed dataset of 45,000 movies, which included the cast, genres, and keywords.

2.1 Running Out of Memory

The first major problem was computer memory. Trying to mix 25 million ratings with complex movie details immediately crashed my computer.

To fix this, I had to be strict about what data to keep. I completely dropped unnecessary tags and focused only on the most important keywords. Also, the movie details were stored in a messy text format. Cleaning this text took a lot of computer power.

The Solution: I created a strict cleaning process that only kept the top three actors, the director, and the main genres. I threw everything else away. I also made sure to constantly clear my computer's memory while processing the data, so it wouldn't crash.

3 Phase 2: Building the Core Engine

A good recommendation engine needs more than one trick. If you only look at the movie details, the system suggests movies that are too similar. If you only look at user habits, it ignores what the movie is actually about. CineIQ uses a mix of both.

3.1 Finding Similar Movies (The *What*)

To find movies with a similar feel, I created a giant text summary for each movie. This summary included its genres, keywords, main actors, and director. I then turned these text summaries into numbers that the computer could understand.

When you pick a movie you like, the system compares its numbers against the whole database. This instantly narrows the 45,000 movies down to a highly relevant top 50 list.

3.2 Understanding User Taste (The *Who*)

Once I had 50 similar movies, I needed to know if *you* would actually like them. I used a math technique called Singular Value Decomposition. This technique looks at the massive grid of user ratings to find hidden patterns in what people enjoy.

This math model scores those top 50 movies from 0.5 to 5.0 based on your past ratings. This sorts the list perfectly to match your personal taste.

4 Phase 3: Adding Audience Sentiment

This is where standard systems stop, and where CineIQ takes a step further. Just because a movie shares keywords with *The Godfather* and people with your taste watched it, doesn't mean it is actually a good movie. I needed the system to understand critical reviews.

4.1 Training AI in the Cloud

I decided to use a powerful AI model that understands language, called DistilBERT. Because training this AI on my laptop was impossible, I moved to a cloud server using Google Colab, which has special hardware for this kind of work.

I trained the AI on 50,000 movie reviews. I didn't just want it to say "good" or "bad"; I wanted a specific number to show exactly how people felt. Once trained, I downloaded this AI back to my computer.

When CineIQ finds a movie, the AI checks its reviews. If the plot or reviews are very negative, the AI lowers the final score. If it is highly praised, the AI boosts the score.

$$Final_Score = Math_Prediction \times Audience_Sentiment_Multiplier \quad (1)$$

5 Phase 4: Making the System Fast

At first, the whole process, including the text matching, math predictions, and AI review checks, was stuffed into the main user screen. This made the app very slow because it had to load heavy files every time you clicked a button.

To fix this, I split the app into two separate parts:

- **The Backend:** I created a separate server that loads the heavy AI models into memory just once when it starts. It waits quietly in the background. When you ask for a movie, it does all the hard math in milliseconds and sends the top 10 list back.
- **The Frontend:** The user screen is now very light and fast. It simply takes your clicks, sends a quick request to the brain, and displays the results beautifully.

6 Phase 5: Explaining the Choices

The final piece of the puzzle was solving the hidden algorithm problem. I wanted you to feel understood by the engine.

First, I added a dynamic chart on the dashboard. By looking at all the movies you rated highly, the chart visually maps out your favorite themes, keywords, and genres. This proves the system knows what you like.

Second, I added a clear explanation for every single result. Instead of just listing the Top 10 movies, CineIQ prints a simple sentence telling you exactly why a movie was chosen, like:

"Why this? It shares strong themes with 'The Matchmaker'. Users similar to you rated it highly (Math Prediction: 3.63), and your custom AI model saw that audiences loved it."

7 Conclusion

Building CineIQ was a huge learning experience. Moving from a crashing laptop to a fast, cloud-trained AI system took a lot of trial and error.

The final project proves that streaming platforms do not need to hide their algorithms. By combining clever math with AI that understands language, and wrapping it in a clear user interface, we can build recommendation engines that respect your choices and help you find movies you truly love.